# SOFTWARE ENGINEERING LAB

Project Documentation: HOTEL MANAGEMENT SYSTEM

| **Group Member** | **Enrollment / Roll No.** |
|---|---|
| Sahil Kumar | 20252651044 |
| Sonu Jha | 20252651056 |
| Aakash | 20252651001 |
| Manish Saxsena | 20252651033 |
| Vedansh Raghwanshi | 20252651062 |

| | |
|---|---|
| **Course / Batch:** | MCA (2nd Year) — Software Engineering Lab |
| **Date of Submission:** | 29/10/2026 |

# Experiment 1: Software Requirements Specification (SRS) Framework

**Aim:** To analyze the project domain for a modern Hotel Management System (HMS) and compile a comprehensive Software Requirements Specification outline detailing stakeholders, functionalities, performance parameters, and architectural system boundaries.

## 1.1 Project Stakeholder Matrix

| Stakeholder Entity | Operational Responsibility within HMS | Core System Interactivity |
|---|---|---|
| Guest / Customer | Initiates remote queries, updates personal metadata, processes payments. | External consumer layer interface access. |
| Receptionist / Front Desk | Manages operational live status check-ins, check-outs, room re-allocations. | Internal operations terminal dashboard client. |
| System Administrator | Performs continuous master infrastructure configuration, backup auditing, user provisioning. | Root console full control panel access. |

## 1.2 Draw.io Architecture Diagram: System Context Boundary View

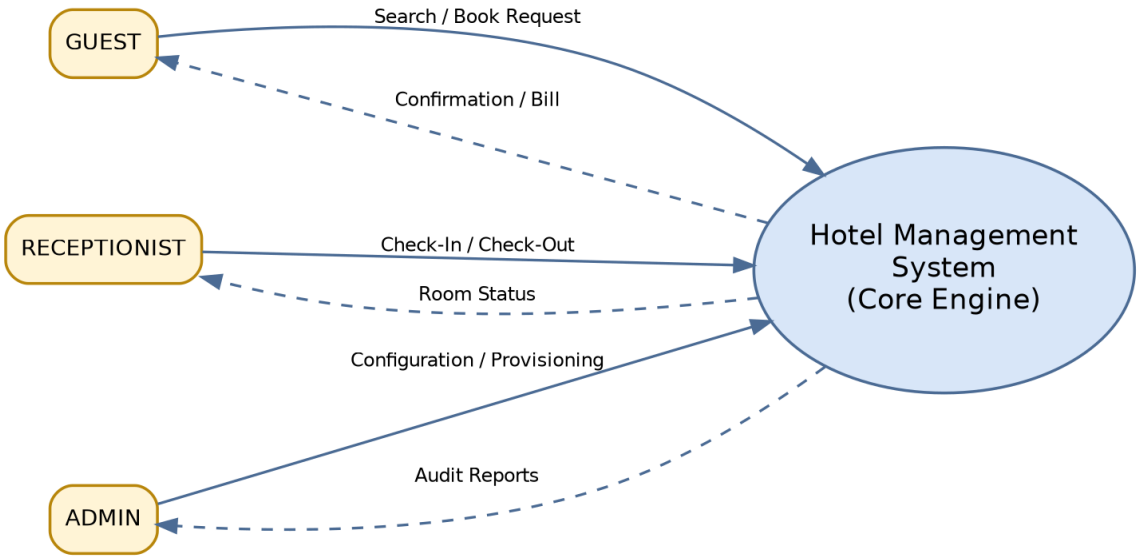



*Figure 1.2: System Context Diagram - Hotel Management System*

# Experiment 2: Software Development Life Cycle (SDLC) Model Optimization

**Aim:** To contrast structural software process engineering models (Waterfall, Spiral, Agile/Scrum, Prototyping) and evaluate their fitness criteria for application deployment within a Hotel Management System framework.

## 2.1 Comparative Analysis Matrix

| SDLC Process Model | Requirement Volatility | Feedback Loop Rate | Risk Containment Tier | HMS Applicability Verdict |
|---|---|---|---|---|
| Waterfall Model | Static / Highly Rigid | Extremely Low | Low Capability | Sub-optimal: Fails with market feature iterations. |
| Prototype Model | Evolutionary / Evolving | High (Pre-launch) | Medium Tier | Good for front-end interface adjustments. |
| Spiral Framework | Highly Adaptable | Periodic / Cycle-based | Very High / Strong | Over-engineered: Exceeds standard system risk bounds. |
| Agile (Scrum Framework) | Fluid / Rapidly Changing | Continuous / Iterative | High Efficiency | Optimized: Excellent for sprint feature updates. |

## 2.2 Draw.io Agile Cycle Flow Diagram

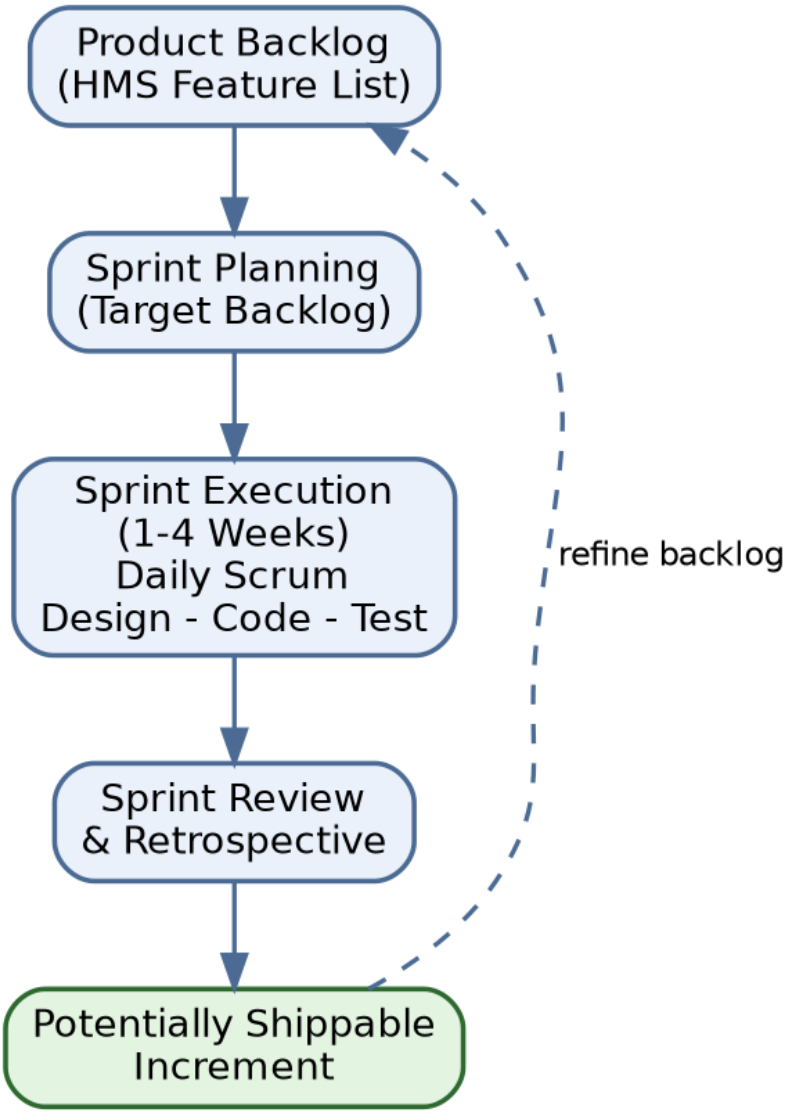



*Figure 2.2: Agile (Scrum) Cycle Flow Diagram*

# Experiment 3: Functional Use Case Engineering Topology

**Aim:** To identify and establish concrete behavioral boundaries for the Hotel Management System through formalized Use Case scenarios, explicitly mapping actor roles and inclusion/extension rules.

## 3.1 Core System Use Case Map

| Primary Actor | Target Functional Use Case | Stereotype Dependency | Pre-Condition Boundary |
|---|---|---|---|
| Guest | Book Room | Includes: <<Authenticate User>> | User session initialized active. |
| Guest | Cancel Booking | Extends: <<Apply Cancellation Fee>> | Existing active booking entry matching account. |
| Receptionist | Check-In Guest | Includes: <<Check Room Availability>> | Assigned reservation record present in queue. |
| Receptionist | Generate Bill/Invoice | Includes: <<Process Payment>> | Checkout processing sequence triggered. |
| Admin | Manage Room Assets | None Base Vector | Root administrative authentication verified. |

## 3.2 Draw.io Use Case Diagram Architecture

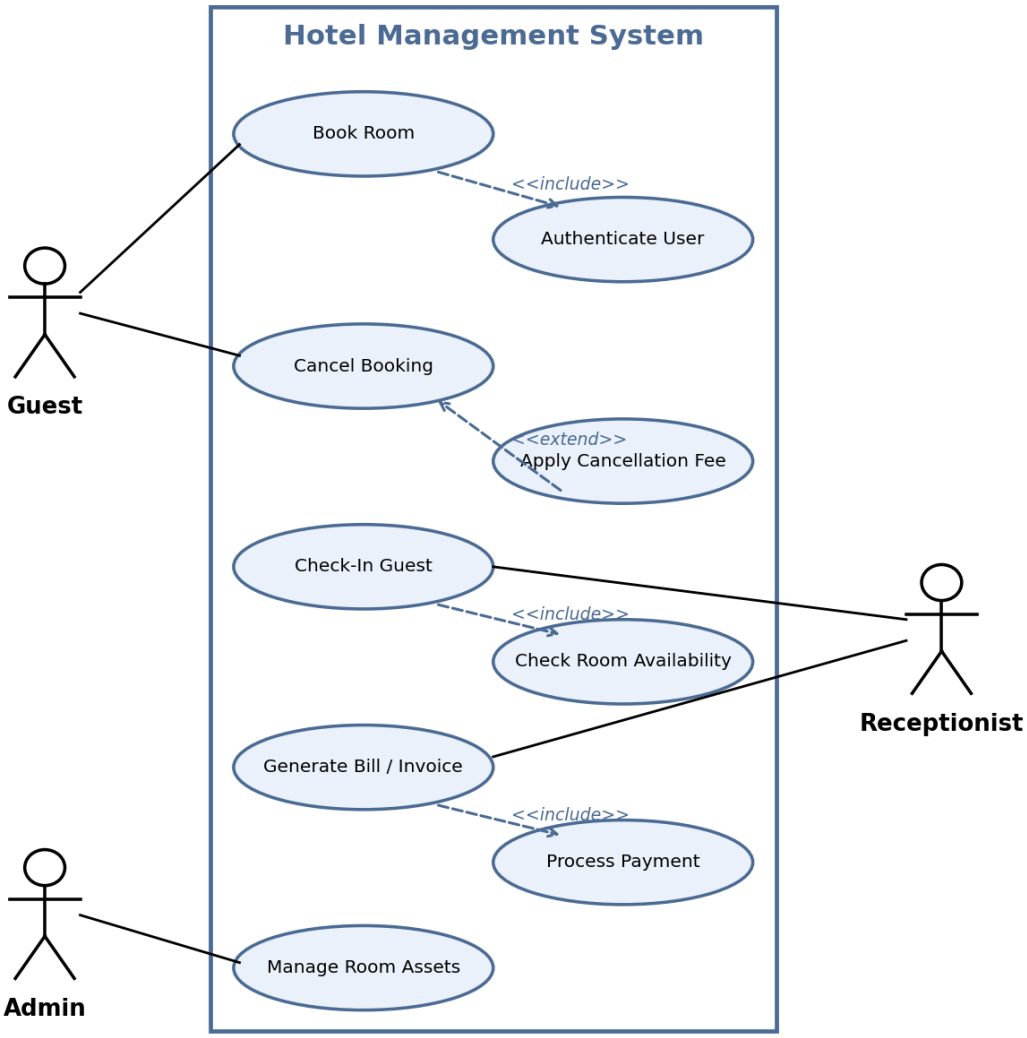



*Figure 3.2: Use Case Diagram - Hotel Management System*

# Experiment 4: Structural Data Flow Architecture (DFD Mapping)

**Aim:** To establish functional transformations and data flow routing within the Hotel Management System, creating Context, Level 0, and Level 1 Data Flow Diagrams.

## 4.1 Data Repository and Entity Identifiers

| Identifier | Structural Data Store / Entity Name | Payload Description Stored |
|---|---|---|
| D1 | Guests Master Ledger | Personal attributes: profile information, contact identifiers, security salts. |
| D2 | Rooms Status Catalog | Structural data: classifications, rates, real-time availability states. |
| D3 | Bookings Central Matrix | Transactional information: schedules, allocations, assignment keys. |
| D4 | Finance Transactions Log | Invoices, settlement records, tokenized logs. |

## 4.2 Draw.io Level 0 Data Flow Diagram Routing Diagram

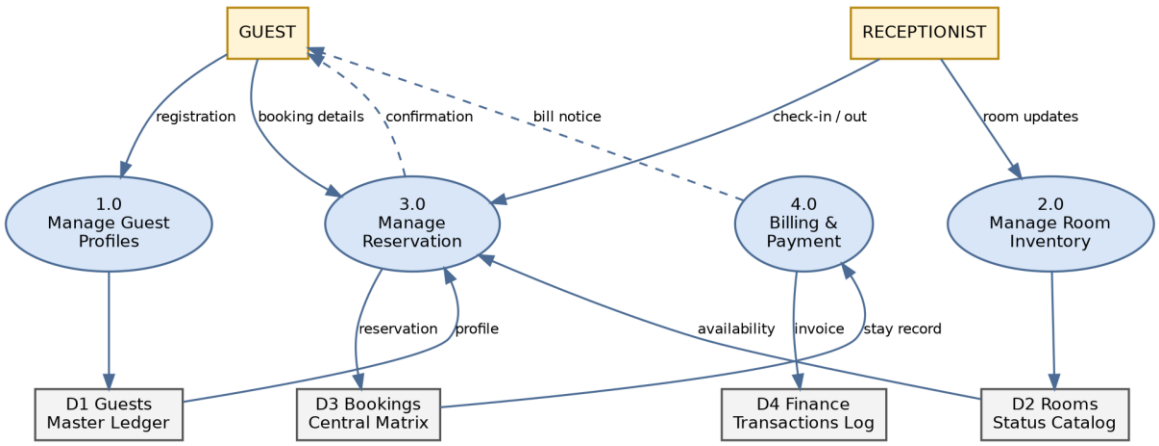



*Figure 4.2: Level 0 Data Flow Diagram*

# Experiment 5: Static UML Class Architecture Blueprint

**Aim:** To define the persistent object data structure of the Hotel Management System through a structural UML Class Diagram, mapping class interfaces, inner fields, data types, and multiplicities.

## 5.1 Class Diagram Relational Mapping Matrix

| Class Origin A | Structural Link Vector | Class Destination B | Multiplicity A | Multiplicity B |
|---|---|---|---|---|
| Guest | Association: makes | Booking | 1 | * (Zero to Many) |
| Booking | Association: maps to | Room | * | 1..* (One to Many) |
| Booking | Composition: generates | Invoice | 1 | 1 (Exactly One) |
| Employee | Generalization (Inherited Base) | Receptionist / Manager | 1 | Sub-class (Is-A) |

## 5.2 Draw.io Static UML Class Layout Architecture

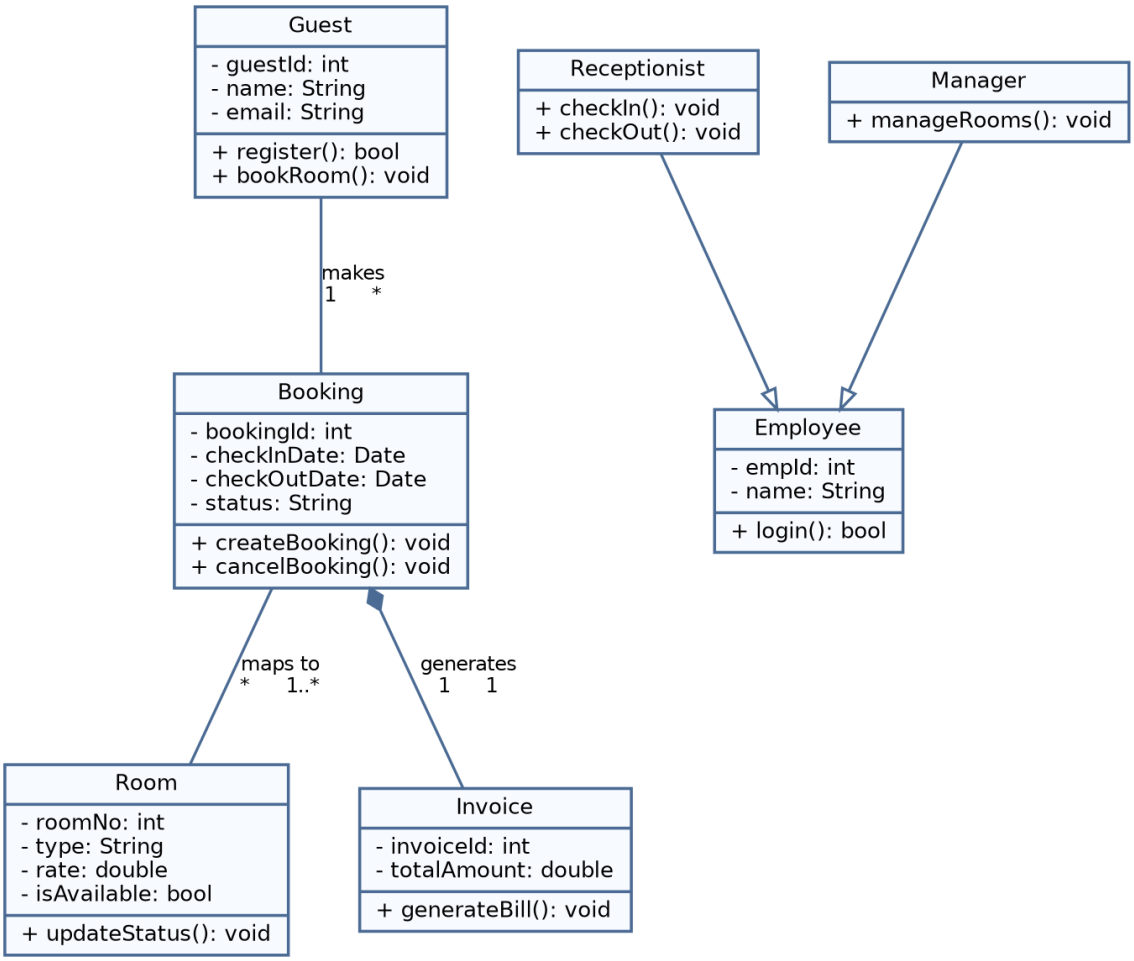



*Figure 5.2: UML Class Diagram*

# Experiment 6: Dynamic Behavior Modeling (UML Sequence Diagram)

**Aim:** To establish runtime communications and message synchronization within the Hotel Management System through an interactive UML Sequence Diagram illustrating the room booking transaction workflow.

## 6.1 Sequential Step Registry

| Step Index | Source Sub-Component | Target System Link | Synchronous Payload / Operation |
|---|---|---|---|
| 1.0 | Guest Actor Client | BookingUI Interface | submitReservationRequest(date, roomType) |
| 2.0 | BookingUI Interface | ReservationManager Engine | verifyAllocationAvailability(date, roomType) |
| 3.0 | ReservationManager | RoomDB Repository | queryRoomStateFlags(date) |
| 4.0 | RoomDB Repository | ReservationManager Engine | Return availability token (Status: Valid) |
| 5.0 | ReservationManager | BookingUI Interface | renderReservationSuccessConfirmation() |

## 6.2 Draw.io Dynamic Sequence Diagram Tracing Execution Timeline

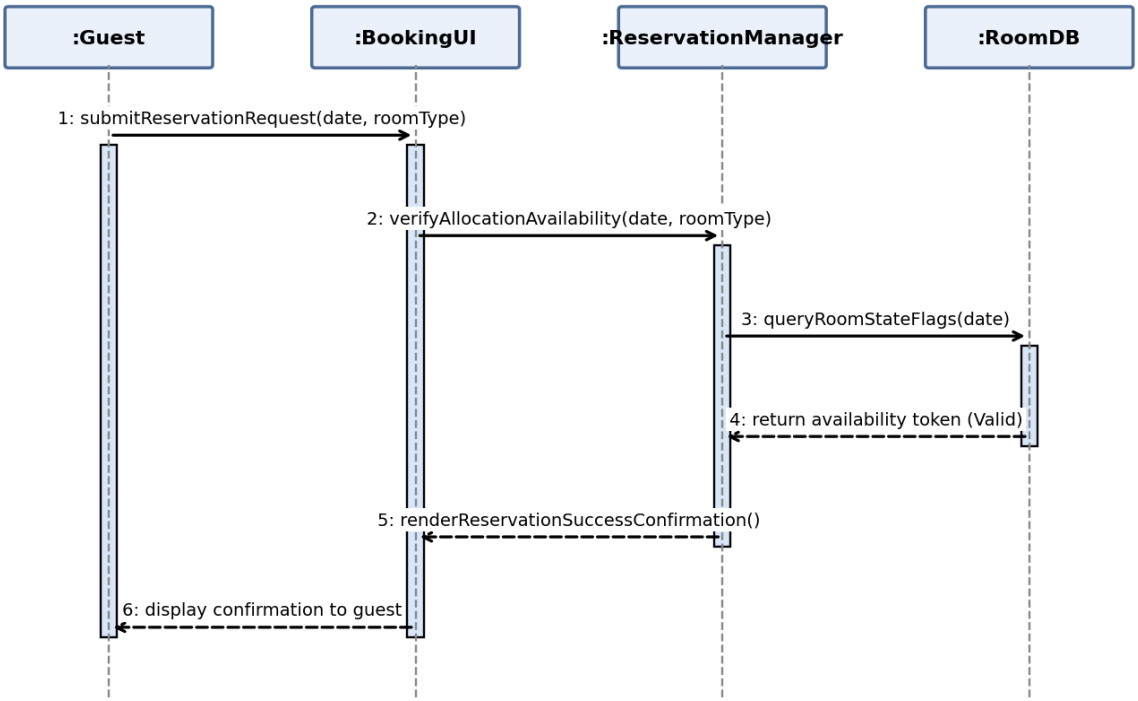



*Figure 6.2: UML Sequence Diagram - Room Booking*

# Experiment 7: Software Project Management Plan (SPMP Metrics)

**Aim:** To formulate standard operational metrics and resource estimates for the Hotel Management System using empirical algorithmic sizing tools (Basic COCOMO) and construct a project milestone schedule.

## 7.1 Sizing Calculations via Basic COCOMO Formula

Given Size Allocation = 32 KLOC (Organic System Project Parameters: a_b=2.4, b_b=1.05, c_b=2.5, d_b=0.38)

1. Computational Formula for Project Effort Calculation:
   Effort = a_b * (KLOC)^(b_b) = 2.4 * (32)^(1.05)
   Resulting Effort = 2.4 * 38.34 = 92.01 Person-Months

2. Computational Formula for Project Development Duration Calculation:
   Duration = c_b * (Effort)^(d_b) = 2.5 * (92.01)^(0.38)
   Resulting Duration = 2.5 * 5.54 = 13.85 Calendar Months

3. Average Optimal Team Staffing Requirements Matrix:
   Staff Required = Effort / Duration = 92.01 / 13.85 = 6.64 Developers (~7 Full-Time Engineers)

## 7.2 Draw.io Project Gantt Timeline Milestone Schedule

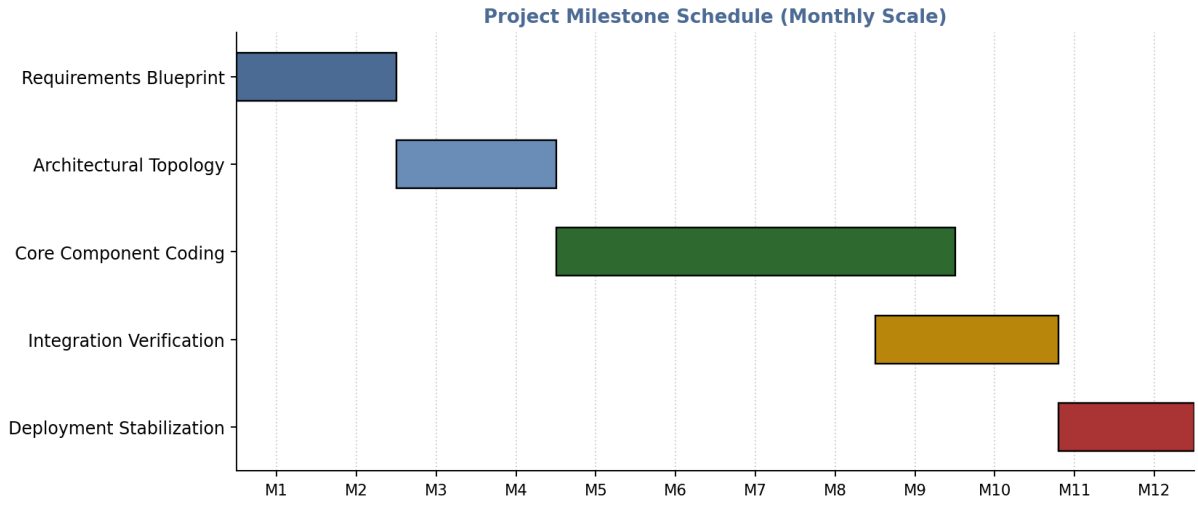



*Figure 7.2: Project Gantt Chart - Milestone Schedule*

# Project Planning Documentation

**Aim:** To document the planning of the Hotel Management System project: scope, team organisation, work breakdown, schedule, risk management and planning parameters, building on the Basic COCOMO estimates of Experiment 7.

## 7A.1 Project Overview and Scope

| Parameter | Description |
|---|---|
| **Project Title** | Hotel Management System (HMS) |
| **Objective** | Automate room search, booking, cancellation, check-in/check-out, billing and room management for a hotel. |
| **In Scope** | Guest registration and login, room availability search, reservation and cancellation, front-desk check-in/out, invoice generation and payment recording, admin room and user management. |
| **Out of Scope** | Third-party channel manager integration, mobile application, loyalty programme. |
| **Process Model** | Agile (Scrum), as justified in Experiment 2. |
| **Effort Estimate** | 92.01 person-months, 13.85 months duration, about 7 developers (Basic COCOMO, 32 KLOC, Experiment 7). |

## 7A.2 Team Organisation and Responsibilities

| Role | Responsibility | Team Member |
|---|---|---|
| **Project Manager / Documentation** | Planning, schedule tracking, SRS and journal compilation, submission. | Sahil Kumar (20252651044) |
| **Requirements Analyst** | Stakeholder analysis, SRS, use case identification. | Sonu Jha (20252651056) |

| Role | Responsibility | Team Member |
|---|---|---|
| **System Designer** | DFD, UML class and sequence diagrams, module decomposition. | Manish Saxena (20252651033) |
| **Developer** | Implementation of UserAuth, Reservation and Finance modules. | Aakash (20252651001) |
| **Tester / QA** | Test planning, test case design and execution, defect reporting. | Vedansh Raghwanshi (20252651062) |

## 7A.3 Work Breakdown Structure (WBS)

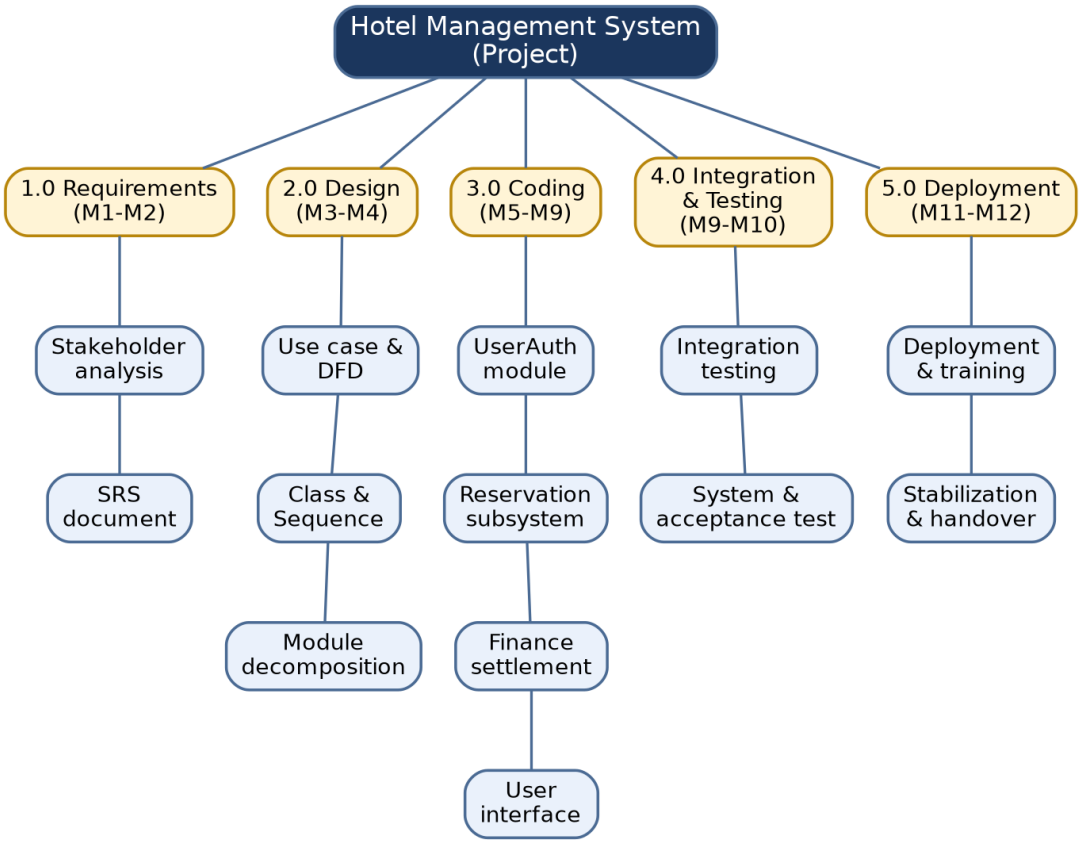



*Figure 7A.3: Work Breakdown Structure of the Hotel Management System*

| Phase | Major Activities | Duration | Deliverable |
|---|---|---|---|
| **1.0 Requirements** | Stakeholder analysis, requirement elicitation, SRS preparation. | M1 - M2 | SRS document |
| **2.0 Design** | Use case, DFD, class and sequence diagrams, subsystem decomposition. | M3 - M4 | Design documents |
| **3.0 Coding** | Implementation of UserAuth, Reservation, Finance modules and user interface. | M5 - M9 | Source code |
| **4.0 Integration and Testing** | Module integration, system testing, defect fixing. | M9 - M10 | Test report |
| **5.0 Deployment** | Deployment, user training, stabilization and handover. | M11 - M12 | Deployed system and user manual |

## 7A.4 Milestone Schedule

| No. | Milestone | Target Month | Completion Criterion |
|---|---|---|---|
| M-1 | SRS baseline approved | End of M2 | Requirements reviewed and frozen for first release. |
| M-2 | Design baseline approved | End of M4 | All UML and DFD diagrams reviewed. |
| M-3 | Core modules coded | End of M9 | Unit tests passed for all modules. |
| M-4 | Integration and system testing complete | End of M10 | All critical and major defects closed. |
| M-5 | Release and handover | End of M12 | Acceptance sign-off obtained. |

## 7A.5 Risk Management Plan

| Risk | Probability | Impact | Mitigation Strategy |
|---|---|---|---|
| **Requirement changes during development** | High | High | Agile sprints with backlog refinement; change control through sprint review. |
| **Double booking due to concurrency** | Medium | High | Transactional database locking; test case TC-HMS-004. |
| **Schedule slippage** | Medium | Medium | Weekly progress review and buffer in the coding phase. |
| **Team member unavailability** | Medium | Medium | Cross-training and shared documentation. |
| **Data loss or security breach** | Low | High | Regular backups, hashed credentials, role-based access. |
| **Integration defects between modules** | Medium | Medium | Low coupling interfaces (Experiment 8) and early integration testing. |

## 7A.6 Planning Parameters

| Planning Item | Decision |
|---|---|
| **Sprint Length** | 2 weeks, within the 1 to 4 week Scrum range of Experiment 2. |
| **Progress Review** | Weekly team meeting with sprint review and retrospective at sprint end. |
| **Configuration Management** | Version control of source code and documents; baselines at each milestone. |

| Planning Item | Decision |
| --- | --- |
| **Quality Assurance** | Peer review of SRS and design; test-driven verification as in Experiment 9. |
| **Note on Scale** | COCOMO gives an industrial-scale estimate; the five-member academic team delivers a reduced-scope first release. |

# Experiment 8: Advanced Modular Architecture Decomposition

**Aim:** To partition the Hotel Management System into discrete structural subsystems, maximizing internal module cohesion while minimizing cross-component coupling dependencies.

## 8.1 Cohesion and Coupling Structural Specification

| Subsystem Component | Internal Functional Mandate | Cohesion Matrix Classification | Coupling Interface Control |
|---|---|---|---|
| UserAuth Module | Handles security validation tokens, session lifecycle controls, and identity mapping. | High Functional Cohesion | Low Data Coupling (Exposes isolate identity checks) |
| ReservationSubsystem | Manages scheduling pipelines, allocations, inventory availability lookups. | High Sequential Cohesion | Medium Data Coupling (Interacts with room state storage) |
| FinanceSettlement | Processes invoices, line item tracking, and transaction settlement. | High Informational Cohesion | Low Message Coupling (Driven by tokenized triggers) |

## 8.2 Draw.io Subsystem Architecture Component View

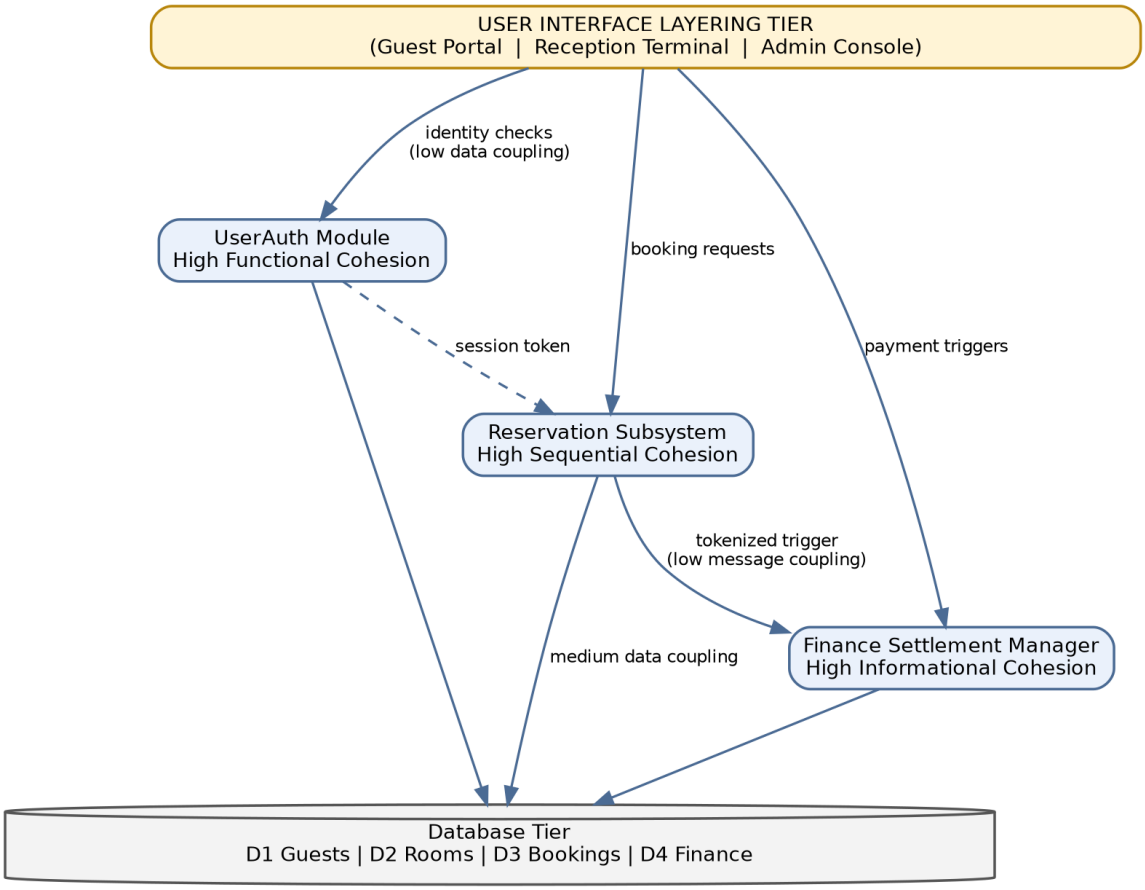



*Figure 8.2: Subsystem Architecture Component View*

# Experiment 9: Comprehensive Black-Box Verification & Validation

**Aim:** To design structured, functional verification conditions for the Hotel Management System using black-box testing methodologies, validating both positive and negative operational parameters.

## 9.1 Comprehensive System Test Case Register

| Test Identifier | Functional Sizing Vector | Input Evaluation Payload | Observable Expected System Output | Validation Strategy |
|---|---|---|---|---|
| TC-HMS-001 | User Login Profile Verification | Valid Username + Valid Cryptographic Password | System profile dashboard initializes successfully. | Positive Test Baseline |
| TC-HMS-002 | User Login Profile Verification | Valid Username + Incorrect Cryptographic Password | Security alert triggered, authentication rejected. | Negative Test Boundary |
| TC-HMS-003 | Room Allocation Request Engine | Target Room Available (Status Flags Open) | Allocation locked, confirmation token generated. | Positive Test Baseline |
| TC-HMS-004 | Double Reservation Conflict Prevention | Request Room on dates already fully booked | Transaction rolls back, error message displayed. | Negative Test Boundary |

## 9.2 Draw.io Test Pipeline Pipeline Flowchart

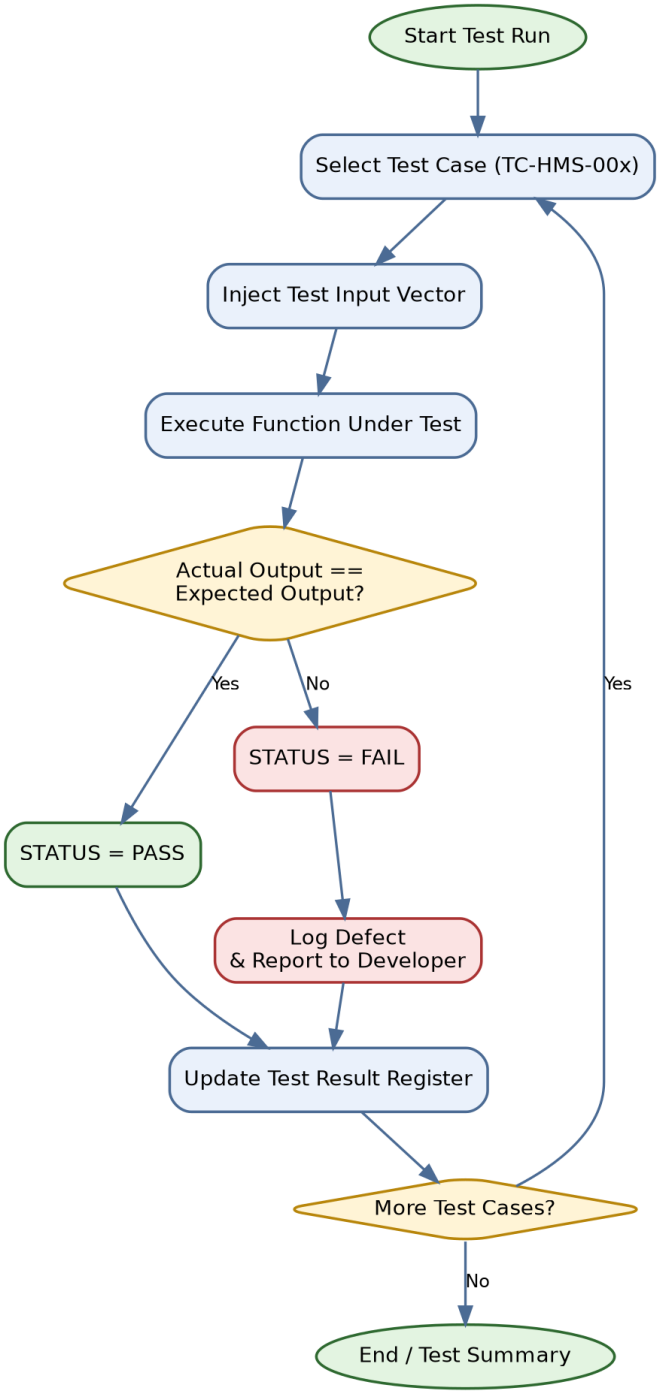



*Figure 9.2: Black-Box Test Execution Flowchart*

# Testing Documentation

**Aim:** To document the complete test plan, test cases, traceability, test design techniques and defect handling for the Hotel Management System, extending the black-box verification of Experiment 9.

## T.1 Test Plan

| Item | Description |
|---|---|
| **Test Objective** | Verify that every functional requirement of HMS behaves as specified, for valid and invalid inputs. |
| **Features to be Tested** | Login, guest registration, room search, booking, cancellation, check-in/out, billing and payment, admin room management, access control. |
| **Features not Tested** | Performance and load testing, third-party payment gateway internals. |
| **Test Approach** | Black-box testing using equivalence partitioning and boundary value analysis, at unit, integration, system and acceptance levels. |
| **Test Environment** | Web browser client, application server and relational database holding stores D1 to D4. |
| **Entry Criteria** | Module coded, unit tests prepared, test data available. |
| **Exit Criteria** | All test cases executed, no open critical or major defects. |
| **Pass / Fail Rule** | A test passes when actual output equals the expected output; otherwise it is logged as a defect. |

## T.2 Levels of Testing (V-Model)

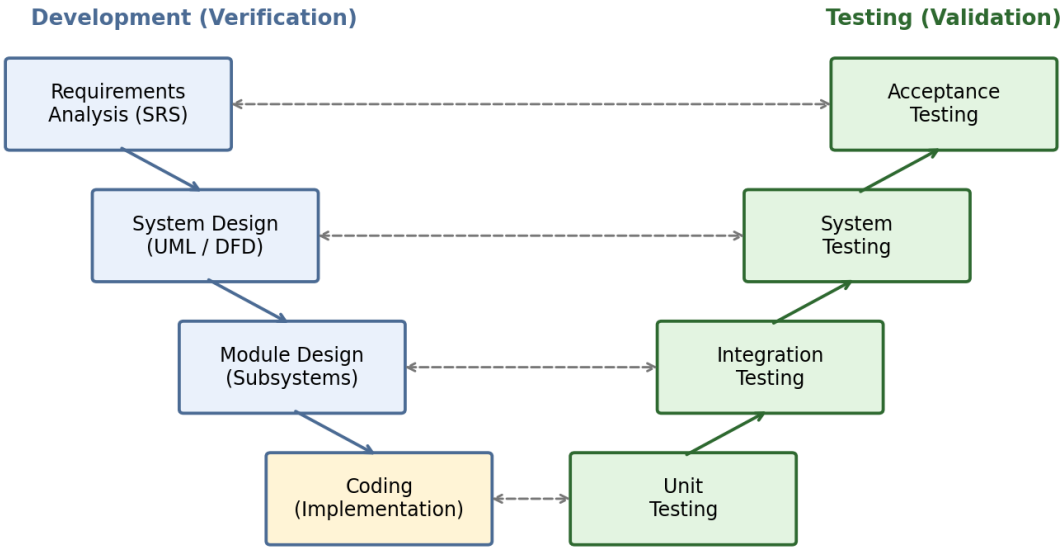



*Figure T.2: V-Model mapping development phases to testing levels*

## T.3 Extended Test Case Register

Test cases TC-HMS-001 to TC-HMS-004 are listed in Experiment 9. The register below extends them to cover all modules.

| Test ID | Module | Input | Expected Output | Type |
|---|---|---|---|---|
| **TC-HMS-005** | Guest Registration | Valid name, email and password | Profile created in D1 and confirmation shown. | Positive |
| **TC-HMS-006** | Guest Registration | Email that is already registered | Error message: account already exists. | Negative |
| **TC-HMS-007** | Room Search | Valid date range and room type | List of available rooms is displayed. | Positive |
| **TC-HMS-008** | Booking | Check-out date earlier than check-in date | Validation error; booking not created. | Negative / Boundary |

| Test ID | Module | Input | Expected Output | Type |
|---|---|---|---|---|
| **TC-HMS-009** | Cancel Booking | Existing active booking | Status set to Cancelled, fee applied as per policy, room released. | Positive |
| **TC-HMS-010** | Check-In | Valid reservation ID entered by receptionist | Room status changes to Occupied. | Positive |
| **TC-HMS-011** | Check-In | Reservation ID not present in system | Error message: reservation not found. | Negative |
| **TC-HMS-012** | Billing | Check-out of a completed stay | Invoice total equals room rate multiplied by nights plus extra charges. | Positive |
| **TC-HMS-013** | Payment | Payment attempt that fails | Invoice remains Unpaid and retry is allowed. | Negative |
| **TC-HMS-014** | Room Management | Admin adds a room with an existing room number | Request rejected with duplicate error. | Negative |
| **TC-HMS-015** | Room Management | Admin adds a room with valid details | Room stored in D2 and shown as available. | Positive |
| **TC-HMS-016** | Access Control | Guest opens an admin-only page | Access denied and event logged. | Negative / Security |

## T.4 Requirements Traceability Matrix

| Req. ID | Functional Requirement | Covered by Test Cases |
|---|---|---|
| **FR-01** | User authentication and access control | TC-HMS-001, 002, 016 |

| Req. ID | Functional Requirement | Covered by Test Cases |
|---|---|---|
| **FR-02** | Guest registration | TC-HMS-005, 006 |
| **FR-03** | Room search and availability | TC-HMS-003, 007 |
| **FR-04** | Room booking | TC-HMS-003, 004, 008 |
| **FR-05** | Booking cancellation | TC-HMS-009 |
| **FR-06** | Check-in and check-out | TC-HMS-010, 011 |
| **FR-07** | Billing and payment | TC-HMS-012, 013 |
| **FR-08** | Room management by admin | TC-HMS-014, 015 |

## T.5 Equivalence Partitioning and Boundary Value Analysis

The input limits below are assumed business rules chosen for test design.

| Input Field | Valid Partition | Invalid Partitions | Boundary Values to Test |
|---|---|---|---|
| **Number of nights** | 1 to 30 | Less than 1; more than 30 | 0, 1, 2, 29, 30, 31 |
| **Guests per room** | 1 to 4 | Less than 1; more than 4 | 0, 1, 2, 3, 4, 5 |
| **Password length** | 8 to 16 characters | Fewer than 8; more than 16 | 7, 8, 9, 15, 16, 17 |
| **Stay dates** | Check-out after check-in | Check-out on or before check-in | Same day, next day, previous day |

## T.6 Defect Severity Classification

| Severity | Definition | Example in HMS |
|---|---|---|
| **Critical** | System crash or data corruption; no workaround. | Double reservation of the same room. |
| **Major** | Key function fails but a workaround exists. | Invoice shows wrong total. |
| **Minor** | Function works with small deviation. | Wrong error message text. |
| **Trivial** | Cosmetic issue only. | Label alignment on dashboard. |